Linux Driver II

開發: 高效能計算實驗室

中正大學 資訊工程學系

How to use GPIO

- Shell script
- C program

GPIO (1)

- GPIO (General Purpose Input Output)
- With it you can interact with the environment, connecting up other devices and turning your microcontroller into something useful.

GPIO (2)

- GPIO has two fundamental operating modes: input and output.
- Input lets you read the voltage on a pin, to see whether it's held low or high and deal with that information programmatically.
- Output lets you set the voltage on a pin, again either high or low.

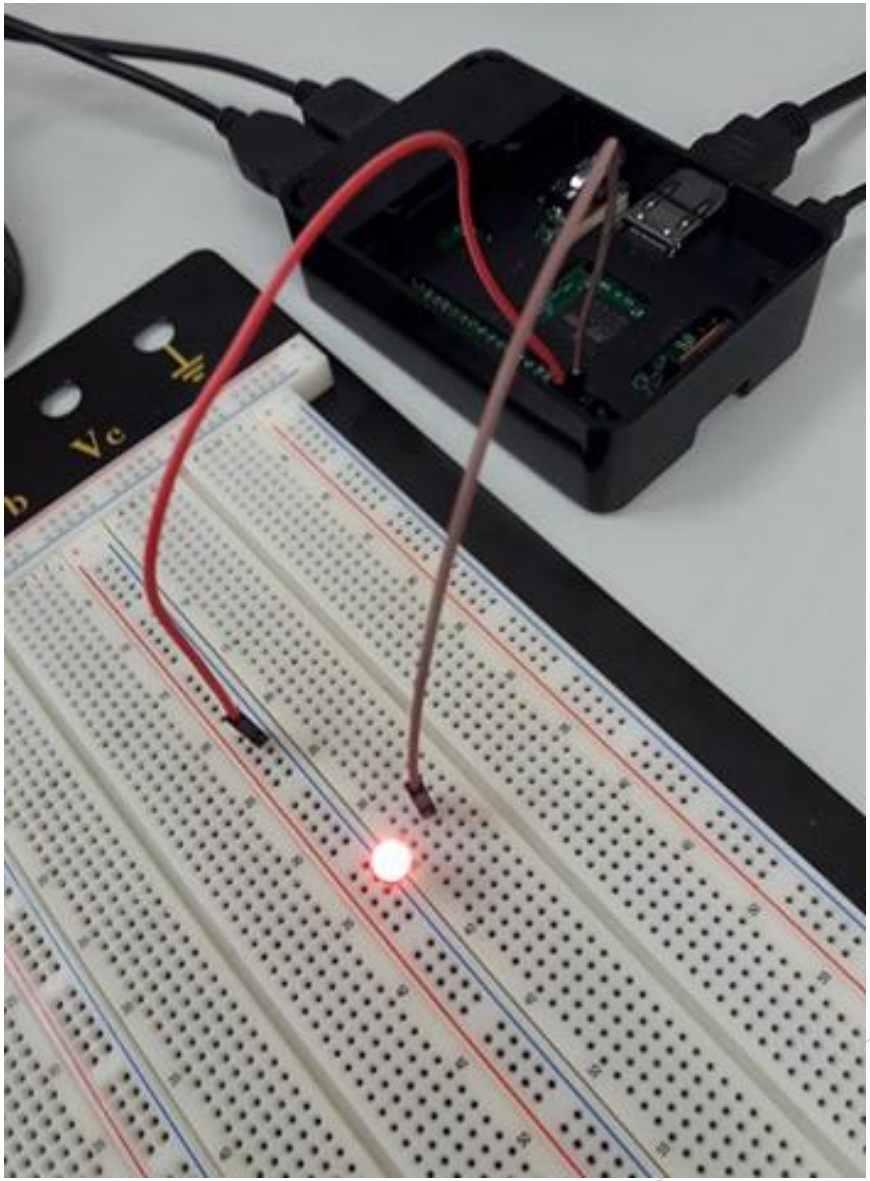
GPIO Header



3.3V PWR	1		2	5V PWR
I2C1 SDA	3		4	5V PWR
I2C1 SCL	5		6	GND
GPIO 4	7		8	Reserved
GND	9		10	Reserved
SPI1 CS0	11		12	GPIO 18
GPIO 27	13		14	GND
GPIO 22	15		16	GPIO 23
3.3V PWR	17		18	GPIO 24
SPI0 MOSI	19		20	GND
SPI0 MISO	21		22	GPIO 25
SPI0 SCLK	23		24	SPI0 CS0
GND	25		26	SPI0 CS1
Reserved	27		28	Reserved
GPIO 5	29		30	GND
GPIO 6	31		32	GPIO 12
GPIO 13	33		34	GND
SPI1 MISO	35		36	GPIO 16
GPIO 26	37		38	SPI1 MOSI
GND	39		40	SPI1 SCLK

實驗使用GPIO4與GND腳位

LED硬體配置



GPIO控制

- sysfs interface
- libgpiod

GPIO的操作 (1): 利用sysfs

- Change to root
- 配置GPIO

```
$ cd /sys/class/gpio
$ echo 4 > export
```

- 產生了一個新的目錄“gpio4”，裡面包含了該I/O的輸出輸入設置等配置文件

- 設置GPIO的輸出/輸入方向

```
$ echo "out" > /sys/class/gpio/gpio4/direction
```

GPIO的操作 (2)

- 設置GPIO的輸出電壓

```
$ echo "1" > /sys/class/gpio/gpio4/value
```

- 關閉GPIO

```
$ echo "4" > /sys/class/gpio/unexport
```

可以看到gpio4已經被刪除

GPIO的操作 (3)

```
$ echo "out" > /sys/class/gpio/gpio4/direction
$ echo "1" > /sys/class/gpio/gpio4/value
```



```
$ echo "high" > /sys/class/gpio/gpio4/direction
```

GPIO的操作: 使用C (1)

- 需由root身份執行程式
- 配置GPIO

```
p = fopen("/sys/class/gpio/export","w");
fprintf(p,"%d",4);
fclose(p);
```

- 產生了一個新的目錄 “gpio4” ，裡面包含了該I/O的輸出輸入設置等配置文件

GPIO的操作: 使用C (2)

- 設置GPIO的輸出/輸入方向

```
p = fopen("/sys/class/gpio/gpio4/direction","w");
fprintf(p,"out");
fclose(p);
```

GPIO的操作: 使用C (3)

- 設置 GPIO的輸出電壓

```
p = fopen("/sys/class/gpio/gpio4/value","w");
fprintf(p,"%d",1);
fclose(p);
```

- 關閉GPIO

```
p = fopen("/sys/class/gpio/unexport","w");
fprintf(p,"%d",4);
fclose(p);
```

可以看到gpio4被刪除

GPIO的操作: 利用libgpiod

- C library and tools for interacting with the linux GPIO character device (e.g., /dev/gpiodchip0)
- 安裝所需軟體
  - gpiod: command-line tools
  - libgpiod-dev: C library
- 安裝Raspberry Pi OS (才能使用apt)

```
$ sudo -s
$ apt install libgpiod-dev gpiod
```

Command tools (1)

```
$ gpioinfo
```

```
user@raspberrypi:~ $ gpioinfo
gpiochip0 - 54 lines:
        line   0:      "ID_SDA"       unused   input  active-high
        line   1:      "ID_SCL"       unused   input  active-high
        line   2:       "GPIO2"       unused   input  active-high
        line   3:       "GPIO3"       unused   input  active-high
        line   4:       "GPIO4"       unused   input  active-high
```

Command tools (2)

命令	作用	範例	說明
gpiodetect	列出所有的GPIO控制器	gpiodetect	列出所有的GPIO devices
gpioinfo	列出GPIO控制器的接腳資訊	gpioinfo gpiochip0 Or gpioinfo 0	列出gpio0控制器接腳資訊
gpioset	設置GPIO	gpioset 0 4=0	设置gpio bank 0 編號4之接脚為低電壓
gpioget	獲取GPIO接腳狀態	gpioget 0 4	獲取gpio bank 0 編號4之接腳狀態
gpiomon	監控gpio的狀態	gpiomon 0 4	監控gpio bank 0 編號4之接腳狀態

C Library

Simple high-level routines for straightforward GPIO manipulation without the need to use the gpiod_* structures or to keep track of resources.

```
#include <gpiod.h>
...


 struct gpiod_chip *chip;        //GPIO控制器
 struct gpiod_line *line;        //GPIO接腳


 /* 獲取GPIO控制器 */
 chip = gpiod_chip_open("/dev/gpiochip0");
 if (chip == NULL) {
    printf("gpiod_chip_open error\n");
    return -1;
 }
```

```
/* 獲取 GPIO 接腳 */
line = gpiod_chip_get_line(chip, 4);
if (line == NULL) {
    printf("gpiod_chip_get_line error\n");
    gpiod_line_release(line);
    gpiod_chip_close(chip);
}

/* 設置GPIO為輸出模式 */
ret = gpiod_line_request_output(line,"led",0);
if (ret < 0) {
    printf("gpiod_line_request_output error\n");
    gpiod_chip_close(chip);
}

gpiod_line_set_value(line, 1);
```

Default value: initial line value

```
//設置GPIO為輸入模式
ret = gpiod_line_request_input(line, “led”);
if (ret < 0) {
    printf("gpiod_line_request_input error\n");
    gpiod_chip_close(chip);
}

gpiod_line_get_value(line);
```

Compilation

```
$ gcc test.c -o test -lgpiod
```

DEMO

- 請同學寫一個 C 程式，讓LED燈可以持續閃爍